

TP02

Knowledge Check Keystone — Questions et corrigé

Lab 2 · 15 questions, corrections et notes à retenir

CONTENU DU DOCUMENT

- 01** Questions, corrigé et notes à retenir (15 questions)
- 02** Synthèse : codes HTTP et points clés

MODULE

Infrastructures cloud natives

DOCUMENT

Questions · corrigé · notes

VERSION

Octobre 2026

QCM Questions, corrigé et notes à retenir

Une seule bonne réponse par question, sauf mention contraire. La bonne réponse est cochée (✓), suivie de sa justification puis d'une note à retenir.

1. Pourquoi Keystone est-il toujours contacté en premier par la CLI `openstack` ?

- ☐ A) Il héberge les machines virtuelles
- ✓ **B) Il fournit le jeton et le catalogue des services**
- ☐ C) Il répartit la charge entre les services
- ☐ D) Il stocke les images des VM

Correction

Le client ne connaît que `OS_AUTH_URL` ; Keystone lui donne un jeton et l'adresse des autres services

À retenir

Tout commence par Keystone : le client ne connaît que `OS_AUTH_URL` . Il s'authentifie, reçoit un jeton **et** le catalogue (les adresses des autres services), puis seulement appelle Nova, Glance, etc.

2. Que contient la colonne `id` de `openstack token issue` ?

- ☐ A) L'identifiant de l'utilisateur
- ☐ B) L'identifiant du projet
- ✓ **C) Le jeton lui-même**
- ☐ D) Le hash SHA256 du jeton

Correction

L' `id` d'un jeton est la chaîne du jeton (`gAAAAAB...`), utilisée telle quelle dans `X-Auth-Token`

À retenir

La colonne `id` est le jeton lui-même (chaîne `gAAAAAB...`), envoyé dans l'en-tête `X-Auth-Token` . Il prouve votre identité jusqu'à expiration : il ne se partage pas.

3. Vrai ou faux : la CLI `openstack` réutilise le même jeton pour toutes les commandes d'une session.

- ☐ Vrai
- ✓ **Faux**

Correction

Chaque commande s'authentifie et obtient un nouveau jeton

À retenir

Un jeton par commande : la CLI s'authentifie à chaque appel. Pour réutiliser un jeton, il faut le passer explicitement (exemple du TP : `export TOKEN=...` puis `curl`).

4. Dans la sortie de `openstack catalog list`, l'URL de `nova_legacy` contient l'ID réel du projet, alors que `openstack endpoint list` affiche `$(project_id)s`. Pourquoi ?

- ✓ A) Le catalogue est généré pour le projet du jeton, avec les variables remplacées
- ☐ B) `endpoint list` affiche une version obsolète du catalogue
- ☐ C) `catalog list` interroge directement Nova
- ☐ D) Les deux commandes contactent des Keystone différents

Correction

`endpoint list` affiche les modèles enregistrés ; le catalogue est construit pour le projet du jeton

À retenir

`endpoint list` montre les **modèles** d'URL (`$(project_id)s`) ; `catalog list` montre le catalogue **résolu** pour le projet du jeton, avec le vrai ID.

5. Tous les services DevStack utilisent la même adresse IP. Comment le serveur sait-il à quel service transmettre une requête ?

- ☐ A) Grâce au jeton
- ✓ B) Grâce au chemin de l'URL (`/identity` , `/compute` , `/image` ...)
- ☐ C) Grâce à l'en-tête `User-Agent`
- ☐ D) Grâce au code de statut HTTP

Correction

Apache joue le rôle de reverse proxy et redirige selon le préfixe du chemin

À retenir

Même IP, même port : c'est le **chemin de l'URL** qui désigne le service (`/identity` , `/compute` , `/image`). Apache joue le rôle de reverse proxy.

6. Dans une sortie `--debug`, vous lisez `X-Auth-Token: {SHA256}3f9a...`. Que signifie cette ligne ?

- ☐ A) Aucun jeton n'a été envoyé
- ☐ B) Le jeton est chiffré sur le réseau
- ✓ C) Un jeton a été envoyé ; le log affiche son empreinte pour ne pas l'exposer
- ☐ D) Le jeton a expiré

Correction

Le client masque le jeton par son hash dans les logs ; le vrai jeton part en clair dans l'en-tête

À retenir

Dans `--debug`, le log affiche l'empreinte `{SHA256}...` du jeton, pas sa valeur. En revanche, le vrai jeton circule dans l'en-tête HTTP, d'où l'importance de HTTPS en production.

7. Candace existe, son mot de passe est correct et `team-a` est son projet par défaut. Pourtant `openstack --os-project-name team-a token issue` renvoie 401. Pourquoi ?

- ☐ A) Son mot de passe a expiré
- ✓ **B) Elle n'a aucun rôle sur `team-a`**
- ☐ C) `team-a` est désactivé
- ☐ D) Le domaine `Default` n'existe pas

Correction

Un jeton est délivré pour un projet seulement si l'utilisateur a un rôle sur ce projet

À retenir

Pas de rôle sur un projet = pas de jeton pour ce projet : **401** renvoyé par Keystone, même avec un bon mot de passe.

8. Ian a le rôle `reader` sur `team-a`. Que se passe-t-il s'il tente de créer une VM dans `team-a` ?

- ☐ A) 401 renvoyé par Keystone
- ✓ **B) 403 renvoyé par Nova**
- ☐ C) 404 renvoyé par Nova
- ☐ D) La VM est créée

Correction

Jeton valide (Keystone OK), mais la règle de création exige `role:member`

À retenir

401 = Keystone refuse l'authentification ; **403** = le service sait qui vous êtes mais la règle d'accès vous refuse l'action. Ici : jeton valide, rôle `reader` insuffisant → 403.

9. Baron (`member` sur `team-b`) fait `openstack server show` avec l'ID de `vm-candace` (projet `team-a`). Quel code obtient-il, et pourquoi ?

- ☐ A) 403 : il n'a pas le bon rôle
- ✓ **B) 404 : pour son projet, la ressource n'existe pas**
- ☐ C) 409 : la VM est dans un état incompatible
- ☐ D) 200 : `member` peut voir toutes les VM

Correction

Les services ne révèlent pas l'existence des ressources d'un autre projet

À retenir

Un service ne révèle pas l'existence des ressources d'un autre projet : vous recevez **404** (« n'existe pas pour vous »), pas 403.

10. La règle de création et d'arrêt d'une VM est `"role:member and project_id:%(project_id)s"` . Que vérifie la condition `project_id:%(project_id)s` ?

- ☐ A) Que l'utilisateur a un projet par défaut
- ✓ **B) Que le projet du jeton est le même que le projet de la ressource visée**
- ☐ C) Que le projet existe dans le catalogue de services
- ☐ D) Que l'utilisateur a créé lui-même la ressource

Correction

`%(...)s` = attribut de la ressource ; la partie gauche (`project_id:`) vient du jeton

À retenir

Dans une règle `role:member and project_id:%(project_id)s` , la partie gauche vient du **jeton** et la droite de la **ressource**. Il faut le bon rôle **et** le même projet.

11. Admin exécute `openstack server list` sans option et ne voit pas `vm-candace` . Pourquoi ?

- ☐ A) Le rôle `admin` ne permet pas de voir les VM des autres projets
- ✓ **B) Son jeton est lié au projet `admin` , et Nova liste par défaut les VM du projet du jeton**
- ☐ C) `vm-candace` est arrêtée
- ☐ D) Admin doit d'abord obtenir le rôle `member` sur `team-a`

Correction

Le rôle donne le droit d'élargir la liste, mais le filtre par défaut reste le projet du jeton

À retenir

Le rôle `admin` ne change pas le périmètre par défaut : le jeton reste lié à un projet et les listes sont filtrées sur ce projet. Pour élargir : `--all-projects` .

12. Candace a le rôle `member` sur `team-a` , et pourtant `openstack server list --all-projects` lui est refusé, alors qu'admin peut l'utiliser. Pourquoi ?

- ☐ A) Candace doit d'abord obtenir le rôle `member` sur tous les projets
- ✓ **B) La règle exige le rôle `admin` , sans alternative `member` et sans condition de projet**
- ☐ C) Son jeton n'est pas lié au projet `admin`
- ☐ D) L'option `--all-projects` est réservée aux utilisateurs du domaine `Default`

Correction

La règle `get_all_tenants` renvoie à `rule:context_is_admin` , soit `role:admin` .

À retenir

`--all-projects` exige `role:admin` (via `context_is_admin`), sans alternative `member` et sans condition de projet.

13. La commande `openstack role assignment list --names` affiche :

Role	User	Project	Domain	Inherited
member	candace	team-a		False
member	baron	team-b		False

Candace, baron, `team-a` et `team-b` appartiennent tous au domaine `Default`. Candace peut-elle créer une VM dans `team-b` ?

- ☐ A) Oui, car candace et `team-b` sont dans le même domaine
- ☐ B) Oui, car candace a le rôle `member`, qui autorise la création de VM
- ✓ ☒ C) Non, car son rôle `member` ne s'applique qu'au projet `team-a`
- ☐ D) Non, car seul baron peut créer des VM dans `team-b`

Correction

`Inherited = False` et la colonne Project remplie signifient que le rôle s'applique uniquement à `team-a`. Candace n'a aucun rôle sur `team-b` : elle ne peut même pas obtenir de jeton pour ce projet (401).

À retenir

Une attribution de rôle = rôle + utilisateur + projet. Avec `Inherited = False`, le rôle ne vaut que pour le projet nommé : ni le domaine, ni un autre projet n'en héritent.

14. Baron voit le réseau `public` alors qu'il ne l'a pas créé. Quelle propriété l'explique ?

- ☐ A) `shared = True`
- ✓ ☒ B) `router:external = External`
- ☐ C) `project_id = team-b`
- ☐ D) `status = ACTIVE`

Correction

Un réseau externe est visible par tous les projets ; `shared` vaut `False`

À retenir

Visibilité et propriété sont indépendantes : un réseau **externe** (`router:external`) est visible par tous les projets sans être `shared`.

15. Vous avez fait `source ~/devstack/openrc admin admin` , **puis** `export OS_CLOUD=candace` .
Quelle identité utilise la commande `openstack server list` ?

- ☐ **A)** Candace, car `OS_CLOUD` est la variable la plus récente
- ✓ **B) Un mélange : les variables** `OS_*` **d'** `openrc` **sont prioritaires sur** `clouds.yaml`
- ☐ **C)** Aucune : la commande échoue toujours
- ☐ **D)** Admin, car `openrc` désactive `clouds.yaml`

Correction

Les variables `OS_*` deviennent les valeurs des options et écrasent celles du profil

À retenir

Les variables `OS_*` d' `openrc` écrasent le profil `clouds.yaml` . Avant de changer d'identité avec `OS_CLOUD` ou `--os-cloud` , videz ces variables (`unset`).

Synthèse L'essentiel de Keystone à retenir

Codes HTTP rencontrés dans le TP

Code	Qui répond ?	Signification	Exemple du TP
401	Keystone	Authentification refusée ou pas de jeton pour ce projet	Candace sans rôle sur le projet visé
403	Le service (Nova, Keystone...)	Identité connue, mais la règle d'accès refuse l'action	Ian (<code>reader</code>) crée une VM ; Candace liste les utilisateurs
404	Le service	Ressource invisible pour le projet du jeton	Baron consulte <code>vm-candace</code>

Les points clés

- **Keystone d'abord** : authentification, jeton, catalogue, puis appel du service.
- **Un jeton = une identité + un projet** : sans rôle sur un projet, pas de jeton pour ce projet.
- **Rôle et projet se vérifient ensemble** : la règle compare le rôle du jeton et le projet de la ressource.
- **Isolation par projet** : une ressource d'un autre projet est invisible (404), pas interdite (403).
- `admin` ≠ **visibilité totale** : le jeton reste lié à un projet ; `--all-projects` est réservé au rôle `admin`.
- **Ordre de priorité des identités** : options de la CLI, puis variables `OS_*`, puis `clouds.yaml`.